CotejAI — Reporte técnico final

Agente autónomo de cuentas por pagar (three-way matching CFDI 4.0) sobre Grok 4.6 · Reto Agente 2026, Track 1 · Licencia MIT · github.com/TEstebanGQ/CotejIA

1. Problema

Antes de pagar una factura, cuentas por pagar debe confirmar tres cosas: que se pidió (orden de compra del ERP), que llegó (remisión de almacén, a menudo una foto con notas a mano) y que lo facturado coincide (CFDI 4.0). Hacerlo a mano toma de 15 a 25 minutos por expediente. Los errores típicos son faltantes, sobreprecios, IVA mal calculado, tipo de cambio inflado, facturas duplicadas o canceladas y proveedores en la lista 69-B del SAT (EFOS). Pagar a un EFOS, además del dinero, compromete la deducción fiscal del comprador.

2. Qué hace el agente

Cada expediente es una unidad de trabajo con su propio ciclo corto. **El código calcula y Grok decide**:

1. Grok 4.6 elige qué herramientas usar (*function calling*, `tool_choice=required`, sin llamadas paralelas): `leer_remision_imagen` (Grok Vision más un filtro de nitidez laplaciano que rechaza fotos borrosas), `consultar_tipo_cambio_real`, `consultar_sat_en_vivo` (SOAP del SAT con *circuit breaker*, lista 69-B y control de UUID duplicado), `cotejar_caso` (motor determinista con `Decimal`) y `enviar_alerta_reclamo_real` (nota de crédito idempotente).
2. Grok cierra el caso con `cerrar_caso`, y una **compuerta de políticas en código** valida el veredicto: los códigos y los montos deben ser exactamente los del cotejo, y el estado debe estar permitido. Si Grok se equivoca, la compuerta lo rechaza con el motivo y Grok corrige.
3. Las políticas son: `MATCH_OK` → **APROBADO**. Solo `QTY_SHORT` / `PRICE_UP` con fuga ≤ $5,000 MXN → **EN_RECLAMO** (retiene la diferencia y pide la nota de crédito). Cualquier otro caso (`SUPPLIER_69B`, `CFDI_CANCELADO`, `DUPLICATE`, `FX_OVERCHARGE`, `TAX_MISMATCH`, `RECEIPT_UNREADABLE`, `SAT_NO_VERIFICADO`, `SYSTEM_ERROR`) → **ESCALADO** a una persona.

Resiliencia: reintentos con *backoff* y *jitter* ante el gateway LLM. Si Grok no responde, el veredicto sale del motor determinista (`fallback_determinista`). El *circuit breaker* del SAT se abre tras 3 fallos y el caso continúa como `SAT_NO_VERIFICADO`. El cotejo tiene 3 reintentos y, si aun así falla, el caso se escala con `SYSTEM_ERROR` y el motivo; nunca en silencio. La idempotencia en BD impide notas de crédito duplicadas. Persistencia: PostgreSQL 16, o SQLite WAL si no hay Postgres.

Costo bajo por diseño: los PDF digitales de remisión se extraen localmente (nivel 1, $0). Grok Vision solo se usa con fotos. En S3, el 91.7% de las remisiones se resolvió sin llamar al modelo.

3. Evidencia con datos

Datasets (generados con semilla, reproducibles byte a byte con `python datasets/generate.py`): 10 casos S2, 36 casos S3 balanceados y 1,000 casos S4 con *ground truth* (`esperado.json`). Composición de S4: 400 limpios, 250 faltantes, 150 de precio/IVA, 100 de tipo de cambio, 50 de 69-B y 50 cancelados/duplicados. Los RFC 69-B provienen del listado público del SAT, y el tipo de cambio de servicios en vivo.

Corrida de carga S4 (`runs/bitacora_s4.csv`): 1,000 tareas en 12 h 14 min (7 nov 2026, 05:38–17:52 UTC), sin caídas del proceso y con la RAM estable entre 120 y 270 MB. Consumo: 7.85 M tokens de entrada, 178 k de salida, **USD 17.49**. Al **auditar la bitácora contra el ground truth** (`python eval/analizar_bitacora_s4.py`):

Resultado de la corrida	Casos	%
Correcto (estado y códigos iguales a lo esperado)	308	30.9%
Escalado por falla del sistema (el cotejo no corrió, sin códigos)	679	68.0%
Error de criterio (CFDI cancelado aprobado)	11	1.1%

Donde el cotejo sí corrió, el agente acertó en **96.6%** (308/319). La corrida tuvo dos problemas reales que la bitácora original ocultaba, porque marcaba `exito=True` en todo:

- **679 escalamientos silenciosos.** El cotejo lanzaba una excepción bajo carga y el caso se cerraba como ESCALADO sin código ni motivo. La causa más probable, reproducida al analizar: cada conexión a la BD sondeaba Postgres con un timeout de ~4 s y re-ejecutaba `init_db()`, lo que sumaba unos 16 s muertos por caso y contención por el

lock. **Corrección:** sondeo e inicialización cacheados por proceso, 3 reintentos en el cotejo, `SYSTEM_ERROR` con el motivo y `exito=False` en la bitácora.

- **CFDI cancelados aprobados.** El acuse de estatus SAT del expediente nunca se leía. **Corrección:** `CFDI_CANCELADO` desde el acuse o desde la respuesta SOAP, con pago 0 y escalamiento. También se agregó `RECEIPT_UNREADABLE` para remisiones sin firma o sin sello.

Después de las correcciones: el motor determinista acierta **1000/1000** en S4 (57 s en total) y **36/36** en S3. Grok 4.6 de punta a punta resolvió correctamente casos que habían fallado, incluido un CFDI cancelado. Hay 82 pruebas automatizadas y todas pasan. **La corrida de 12 h no se repitió**: la bitácora se conserva sin editar.

4. Viabilidad

- **Costo:** un expediente completo con Grok cuesta en promedio **USD 0.037** (unos 15 k tokens; mediana de 34 s). Un analista cuesta unos USD 5. Los casos que se resuelven en el nivel 1 cuestan $0 de visión.
- **Dinero protegido:** en la corrida, el agente retuvo **$5.46 M MXN** de pagos que no procedían. Con las correcciones, el mismo lote identifica **$16.67 M MXN**. Parte de esa cifra son facturas completas bloqueadas (69-B, canceladas, duplicadas), no solo diferencias.
- **Riesgo controlado:** el agente nunca paga ni reclama fuera de la política. Lo dudoso se escala con el expediente y el motivo. El límite autónomo ($5,000 MXN) es configurable por cliente.
- **Siguiente paso:** conexión con Platica (WhatsApp con el proveedor mediante webhook) y descarga masiva de CFDI reales del SAT para un piloto.

5. Cómo evaluarlo (harness)

```
pip install -r requirements.txt
python main.py audit --dir <carpeta>          # un caso o una carpeta de casos → un JSON por línea
python main.py web                            # API + dashboard en http://localhost:8000
```

Cada caso es una carpeta con `oc.json`, `cfdi.xml` y `remision.json`, `remision.pdf` o `remision.png` (opcionalmente `sat_estatus.json`). Si el caso trae `esperado.json`, la salida incluye la comparación. HTTP: `POST /api/v1/auditar` (un expediente en JSON) y `POST /api/v1/auditar-lote`. Si hay `RETO_KEY` corre con Grok; si no, en modo determinista. El contrato completo está en el README.